



Pashov Audit Group

Paxos Labs Security Review

August 4th 2026 - August 6th 2026

Executive Summary

A time-boxed security review of Paxos Labs was conducted through the Pashov Audit Group bug-hunt program. This report documents 5 issues across the reviewed codebase, including findings reported by Slayer Security.

Project	Paxos Labs
Timeline	August 4th 2026 - August 6th 2026

About

Pashov Audit Group is a security research collective focused on smart-contract and protocol security. This report documents the issues surfaced during a time-boxed review delivered through our competitive bug-hunt program.

METHODOLOGY

Researchers independently reviewed the in-scope code and reported the issues they found. Each submission was then triaged and validated by the judging team; duplicates were merged and severities were finalized. Every accepted issue appears below with its impact and a recommended remediation. The client's response to each issue was then reviewed, and its resolution status - fixed, acknowledged, or invalidated - is recorded with the finding.

DISCLAIMER

A security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where the researchers try to find as many issues as possible. We can not guarantee 100% security after the review. Subsequent reviews, bug bounty programs and on-chain monitoring are strongly recommended. The client's remediation of each issue was reviewed after the hunt; the outcome is recorded in the finding's Resolution section.

Risk Classification

Critical	Loss or compromise of critical assets, or protocol-wide impact. Must be fixed before launch.
High	Significant material loss of assets, or serious harm to a group of users.
Medium	Moderate material loss, or moderate harm, often under conditional assumptions.
Low	Minor impact, limited scope, or requires unlikely conditions to trigger.
Info	No direct security impact - best-practice, code-quality or hardening suggestion.

Scope

GitHub Repository: [paxoslabs/nucleus-boring-vault](https://github.com/paxoslabs/nucleus-boring-vault)

Draft PR / Spike: <https://github.com/paxoslabs/nucleus-boring-vault/pull/324>

Commit Hash to Audit: 0af42540fb25a1e5ea85b7142adb4f3a3b7954d5 (branch nick/equivalent-exchange-umanager)

<https://github.com/paxoslabs/nucleus-boring-vault/pull/332/files>

<https://github.com/paxoslabs/nucleus-boring-vault/pull/335>

Findings

SEVERITY	COUNT
Low	5
Total	5

ID	TITLE	SEVERITY	STATUS
L-01	Deposit and withdrawal fees can use stale gold prices while the gold market is closed	Low	Acknowledged
L-02	Withdrawal fixed fee is inflated by the PAXG premium	Low	Fixed
L-03	Mismatched feed timestamps let the PAXG/XAU oracle return a distorted rate	Low	Acknowledged
L-04	Distributor deposits record share locks and refunds against the wrong address	Low	Acknowledged
L-05	Strategist can leave a lasting token approval that drains vault assets	Low	Fixed

Low Severity

[L-01] Deposit and withdrawal fees can use stale gold prices while the gold market is closed

Low

Acknowledged

SUMMARY

`PaxgXauRateProvider.getRate()` derives the PAXG/XAU rate as PAXG/USD divided by XAU/USD:

$$\text{PAXG/XAU} = \text{PAXG/USD} / \text{XAU/USD}$$

The only freshness guard is that both Chainlink answers were updated within **1 days**:

```
if (block.timestamp - lastUpdatedAtXau > MAX_TIME_FROM_LAST_UPDATE) revert;  
if (block.timestamp - lastUpdatedAtPaxg > MAX_TIME_FROM_LAST_UPDATE) revert;
```

The contract never checks whether the XAU market is currently open. Precious metals spot markets such as XAU have scheduled closures, and Chainlink documents that a timestamp only shows when data was last recorded, not whether the market is active. Their guidance is to rely on market status rather than timestamps (<https://docs.chain.link/data-streams/market-hours>).

Because of this, during a closed gold market the last XAU/USD answer is still accepted as long as its timestamp is under one day old, even though PAXG continues trading on crypto markets over the weekend.

For example, if PAXG/USD and XAU/USD both close Friday at 3300 (PAXG/XAU = 1.00) and PAXG then trades higher over the weekend while XAU/USD keeps returning the stale Friday value, `getRate()` still accepts the old XAU answer and computes the fee from an outdated PAXG/XAU rate.

IMPACT

Fees can be mispriced whenever the gold market is closed but the feed remains within the one-day window.

The withdrawal fee is meant to charge more when PAXG is worth more than one ounce of gold. With a stale XAU/USD value, the fee is computed from the old gold price and users can withdraw PAXG too cheaply, shifting value from remaining vault shareholders to withdrawers.

The deposit fee has the opposite risk. It is meant to charge more when PAXG is worth less than one ounce of gold. A stale XAU value can make PAXG look below peg, so new depositors are overcharged, shifting value from depositors to the vault.

RECOMMENDATION

Do not use the XAU/USD leg for PAXGy fee calculation outside active gold market hours.

Options include blocking deposits and withdrawals during closed XAU market windows, sourcing a feed with explicit market-status support, or adding stricter offchain processing controls so deposits and withdrawals are not processed against stale gold prices.

For a custom approach to disabling a feed during specific hours, the Mento team's [MarketHoursBreaker](https://github.com/mento-protocol/mento-core/blob/develop/contracts/oracles/breakers/MarketHoursBreaker.sol) is a useful reference:

<https://github.com/mento-protocol/mento-core/blob/develop/contracts/oracles/breakers/MarketHoursBreaker.sol>

RESOLUTION

ItsNickBarry Paxos Labs

Acknowledged. At this time it is a product requirement to keep deposits and withdrawals active at all times. The risk of oracle mispricing is mitigated by the directional limit of 1:1 PAXG:XAU valuation. We maintain control over when the swap between these assets takes place and at what price, and we expect any de-peg to be temporary, allowing us to delay execution until market pricing is acceptable.

NishithPat Reviewer

Acknowledged.

ni8mare-acknowledged0xl33-acknowledged

[L-02] Withdrawal fixed fee is inflated by the PAXG premium

LowFixed

SUMMARY

`PaxgyDynamicWithdrawalFeeModule` calculates the dynamic fee and the fixed fee independently from the same gross share amount:

```
uint256 dynamicFee =  
    amount.mulDivUp(effectivePrice - PEG_PRICE, effectivePrice);  
uint256 fixedFee =  
    amount.mulDivUp(FIXED_FEE_BPS, BPS_DIVISOR);  
feeAmount = dynamicFee + fixedFee;
```

Let S be the offered shares, $p > 1$ the XAU market value of one PAXG, and f the fixed-fee rate. Because the accountant redeems at the peg ($1 \text{ share} \rightarrow 1 \text{ PAXG}$), redeeming all S shares would pay $S \times p$ XAU. To pay only the correct S XAU entitlement, just S / p PAXG should be redeemed, so the dynamic fee exists purely to strip the PAXG premium: $\text{dynamicFee} = S - S/p = S \times (p-1)/p$.

The dynamic fee is not an extra economic charge; it removes the premium the pegged accountant would otherwise overpay. Deducting $\text{fixedFee} = S \times f$ independently on top leaves shares redeemed $= S/p - S \times f$, giving a market payout of $p \times (S/p - S \times f) = S \times (1 - f \times p)$.

The effective fixed fee is therefore $f \times p$ rather than f , because each fixed-fee share removed after the dynamic correction reduces the payout by a full PAXG worth p XAU.

IMPACT

Whenever PAXG trades above the peg, withdrawing users pay a higher economic fixed fee than the configured basis-point rate, scaling proportionally with the premium.

For example, with PAXG at 1.2 XAU and a configured 5% fixed fee, the user experiences a 6% economic charge, losing an extra 1% of withdrawal value beyond the advertised rate.

This applies systematically to every withdrawal processed above peg, and becomes material at larger premiums or higher fixed-fee settings.

RECOMMENDATION

Calculate the fixed fee first, then compute the dynamic fee from the shares remaining after the fixed deduction:

```
uint256 fixedFee =  
    amount.mulDivUp(FIXED_FEE_BPS, BPS_DIVISOR);  
uint256 remaining = amount - fixedFee;  
uint256 dynamicFee =
```

```
remaining.mulDivUp(effectivePrice - PEG_PRICE, effectivePrice);  
feeAmount = fixedFee + dynamicFee;
```

RESOLUTION

ItsNickBarry Paxos Labs

Fixed. The fixed fee is now deducted from the amount to which the dynamic fee is applied, reflecting the fact that the fixed fee is retained as PAXGy shares and not subject to value adjustment.

NishithPat Reviewer

Fixed at 23cd220

ni8mare-fixed

fixed-0x133

[L-03] Mismatched feed timestamps let the PAXG/XAU oracle return a distorted rate

Low

Acknowledged

LOCATION

PaxgXauRateProvider:getRate:90-121

SUMMARY

The composite oracle returns the value of one PAXG in XAU by dividing the PAXG/USD price by the XAU/USD price. Correctness depends on both observations representing roughly the same market state.

`getRate()` checks freshness independently for each feed but never compares `lastUpdatedAtPaxg` against `lastUpdatedAtXau`. When the feeds update asynchronously, it computes $\text{PAXG/USD}(t_1) / \text{XAU/USD}(t_2)$ even though the two answers reflect different points in time, as long as each `updatedAt` is individually within the allowed freshness window (under 86,500 seconds).

The skew can pair an observation near the end of its freshness window with a newly updated one, producing a composite rate that never represented any single market state.

IMPACT

During asynchronous feed updates or sharp gold-price movements, the deposit and withdrawal fee modules can consume a materially distorted PAXG:XAU rate while both underlying feeds still pass their individual freshness checks.

This can charge an artificial depeg fee or omit a fee that should apply, overcharging or undercharging depositors and withdrawers and transferring value between transacting users and standing vault shareholders, with inconsistent fee accounting as a result.

EXPLOIT PATH

1. A gold-price move updates the XAU/USD feed to a new value while the PAXG/USD feed still exposes its previous round; both `updatedAt` values remain under 86,500 seconds old.
2. A user submits a PAXG deposit or share withdrawal, causing the fee module to call `PaxgXauRateProvider.getRate()`.

3. `getRate()` accepts both answers independently and divides the new XAU/USD value by the stale PAXG/USD value without rejecting the timestamp gap.
4. The fee module treats that distorted quotient as the current PAXG:XAU price, producing an incorrect fee and a value transfer between the user and vault shareholders.

RECOMMENDATION

Reject observations whose timestamps are too far apart before computing the ratio:

```
(, int256 paxgUsd, , uint256 paxgUpdatedAt,) = PAXG_USD_FEED.latestRoundData();
if (block.timestamp - paxgUpdatedAt > MAX_TIME_FROM_LAST_UPDATE) {
    revert MaxTimeFromLastUpdatePassed(block.timestamp, paxgUpdatedAt);
}
if (paxgUpdatedAt > xauUpdatedAt
    ? paxgUpdatedAt - xauUpdatedAt > MAX_FEED_TIME_SKEW
    : xauUpdatedAt - paxgUpdatedAt > MAX_FEED_TIME_SKEW) {
    revert UnsynchronizedFeeds(paxgUpdatedAt, xauUpdatedAt);
}
rate = (paxgUsd.toUint256() * (10 ** RATE_DECIMALS)) / xauUsd.toUint256();
```

If the deployed feed update mechanism guarantees identical timestamps, a stricter minimal fix is to require exact equality:

```
if (lastUpdatedAtPaxg != lastUpdatedAtXau) {
    revert UnsynchronizedFeeds(lastUpdatedAtPaxg, lastUpdatedAtXau);
}
```

RESOLUTION

ItsNickBarry Paxos Labs

Acknowledged. Because we cannot control when the upstream oracles are updated, introducing checks on the skew between their timestamps would also introduce system downtime, which is not acceptable at this time. The same mitigation that applies to L-1 applies here: the 1:1 PAXG:XAU price limit gives us the option to simply wait out a de-peg event and process our swaps only when market pricing is acceptable.

NishithPat Reviewer

Acknowledged.

ni8mare-acknowledged

0x133-acknowledged

[L-04] Distributor deposits record share locks and refunds against the wrong address

Low

Acknowledged

LOCATION

DistributorCodeDepositor:_deposit:149-166

SUMMARY

A direct Teller deposit is meant to keep three things aligned: the minted share owner, the share-lock owner, and the refundable deposit record. The recipient should receive shares while the Teller records the pending deposit against that same address, so the lock and refund mechanisms operate on the real ownership state.

The distributor breaks this by calling `teller.deposit` without a recipient parameter and then forwarding the shares separately. Inside the Teller, `msg.sender` is `DistributorCodeDepositor`, so shares are minted and state recorded for the intermediary:

```
shares = teller.deposit(depositAsset, depositAmountAfterFees, minimumMint);  
  
ERC20(boringVault).safeTransfer(to, shares);
```

The final recipient ends up with no Teller lock or deposit-history entry, while the stored refund entry points at an address that has already transferred its shares away.

IMPACT

Distributor deposits do not preserve the Teller's share-lock and refund invariants.

A final recipient can receive shares without a corresponding Teller lock. The refund state identifies the intermediary, which no longer owns the shares, so a refund can revert because the Teller tries to burn shares from `DistributorCodeDepositor`.

If the vault's transfer hook enforces the intermediary's lock, the forwarding transfer reverts and distributor deposits cannot complete at all during the lock period.

EXPLOIT PATH

1. Alice calls `DistributorCodeDepositor.deposit(asset, amount, minimumMint, Alice, distributorCode, attestation)`.
2. The distributor calls `teller.deposit`; the Teller records Alice's deposit under `DistributorCodeDepositor` and mints shares to `DistributorCodeDepositor`.
3. The distributor transfers those shares to Alice, but no `shareUnlockTime` or `publicDepositHistory` entry is created for Alice.
4. A permitted refund calls `vault.exit` with `DistributorCodeDepositor` as the share owner; the burn reverts because the distributor already forwarded the shares.

RECOMMENDATION

Add a Teller entry point that accepts the final recipient, mints directly to that recipient, and records the lock/refund state against it. The distributor should call this instead of forwarding shares after a normal deposit.

```
function depositFor(  
    ERC20 depositAsset,  
    uint256 depositAmount,  
    uint256 minimumMint,  
    address to  
) external requiresAuth nonReentrant returns (uint256 shares) {  
    if (isPaused) revert TellerWithMultiAssetSupport__Paused();
```



```

if (!isDepositSupported[depositAsset]) {
    revert TellerWithMultiAssetSupport__AssetNotSupported();
}

shares = _erc20Deposit(depositAsset, depositAmount, minimumMint, to);
_afterPublicDeposit(to, depositAsset, depositAmount, shares, shareLockPeriod);
}

```

```

// DistributorCodeDepositor._deposit
shares = teller.depositFor(
    depositAsset,
    depositAmountAfterFees,
    minimumMint,
    to
);
// Remove the intermediate safeTransfer(to, shares).

```

Alternatively, extend the existing Teller deposit flow with an explicit recipient parameter used consistently for `vault.enter` and `_afterPublicDeposit`, while keeping the existing direct-deposit wrapper for calls where `msg.sender` is the intended recipient.

RESOLUTION

ItsNickBarry Paxos Labs

Acknowledged. The share lock is currently disabled and we do not intend to use it with the PAXGy system.

NishithPat Reviewer

Acknowledged. Although I believe `DistributorCodeDepositor` was not in scope for the audit, but was contextually relevant.

ni8mare-acknowledged

0x133-acknowledged

[L-05] Strategist can leave a lasting token approval that drains vault assets

Low

Fixed

LOCATION

EquivalentExchangeUManager:_enforceNoDanglingApprovals:457-493

SUMMARY

A managed vault action is supposed to never leave a persistent ERC-20 allowance that a spender can use after execution. After the merkle-authorized calls and basket accounting complete, every approval call should be verified to have been reset to zero.

The cleanup only inspects approval calls whose target is a basket token. Calls targeting any other ERC-20 are skipped before their selector and allowance are examined:

```

address target = calls.targets[i];
if (!basketTokens.contains(target)) continue;

```

```
bytes4 selector = bytes4(targetData[0:4]);
if (selector != ERC20.approve.selector && selector != INCREASE_ALLOWANCE_SELECTOR) continue;
(address spender, uint256 amount) = abi.decode(targetData[4:68], (address, uint256));
if (amount != 0 && ERC20(target).allowance(boringVault, spender) != 0) revert
DanglingApproval();
```

Because of the early `continue`, a merkle-authorized batch can establish an allowance over a non-basket token and still succeed. This is an approval-lifecycle gap for every non-basket ERC-20 target, not just assets currently tracked in the accounting basket.

IMPACT

A persistent approval over a non-basket asset lets a spender transfer those vault-held tokens later, outside the execution path.

That transfer never passes through the manager's aggregate-value invariant, per-token delta limits, or approval cleanup, so it causes uncovered loss to vault shareholders. The extraction still applies even when the current UManager interface cannot immediately drain the asset, since the allowance remains usable by the external spender.

EXPLOIT PATH

1. A strategist submits `execute()` with a merkle-valid call to `X.approve(attackerSpender, amount)`, where `X` is held by the vault but absent from `basketTokens`.
2. The approval changes no basket balance, so `_checkAndValueBasket()` and the final value invariant succeed.
3. `_enforceNoDanglingApprovals()` skips `X` because `basketTokens.contains(X)` is false, leaving the allowance nonzero.
4. `attackerSpender` calls `X.transferFrom(boringVault, attackerRecipient, amount)`, extracting the approved non-basket tokens without another `execute()` call.

RECOMMENDATION

Inspect all approval targets rather than filtering by basket membership:

```
function _enforceNoDanglingApprovals(ManageCalls calldata calls) internal view {
    uint256 callsLength = calls.targets.length;
    for (uint256 i; i < callsLength; ++i) {
        address target = calls.targets[i];
        bytes calldata targetData = calls.targetData[i];
        if (targetData.length < 68) continue;

        bytes4 selector = bytes4(targetData[0:4]);
        if (selector != ERC20.approve.selector && selector != INCREASE_ALLOWANCE_SELECTOR)
            continue;

        (address spender, uint256 amount) = abi.decode(targetData[4:68], (address, uint256));
        if (amount != 0 && ERC20(target).allowance(boringVault, spender) != 0) {
            revert DanglingApproval();
        }
    }
}
```

Alternatively, maintain a registry of every ERC-20 the vault may approve or hold and check against that complete registry, rather than using the narrower accounting basket as the cleanup domain.

RESOLUTION

ItsNickBarry

Paxos Labs

Fixed. Non-basket tokens which are approved within a batch are now subject to the dangling approval check, and NatSpec has been added to indicate that, in practice, all vault-held tokens must be registered as basket tokens.

NishithPat

Reviewer

Fixed at commit `ce9c62e`

ni8mare-fixed

fixed-0xl33